



KOHISAR MODEL SCHOOL & COLLEGE

SOFTWARE REQUIREMENTS SPECIFICATION

**Kohisar Model School & College — Student Information, Fee & Payroll
Management System (KMS)**

Document Version 1.0

Reverse-Engineered Specification (prepared from an existing Django codebase)

Prepared: 04 September 2026

IEEE 830 / ISO-29148 aligned structure

Table of Contents

1. Introduction
2. Overall Description
3. System Architecture
4. Functional Requirements
 - 4.1 Authentication & Access Control
 - 4.2 School Configuration Module
 - 4.3 Student Information Module
 - 4.4 Family / Household Module
 - 4.5 Fee & Finance Management Module
 - 4.6 Teacher & Payroll Module
 - 4.7 Accounts / Cashbook Module
 - 4.8 Certificates & Printable Documents
 - 4.9 Dashboard & Reporting
5. Data Model (Database Design)
6. External Interface Requirements
7. Non-Functional Requirements
8. Business Rules Summary
9. Appendix A — URL / Route Map
10. Appendix B — Technology Stack & Setup

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes, in full rebuildable detail, the requirements for the Kohisar Model School & College Management System (KMS) — a web-based School Management, Fee Accounting and Payroll System built for Kohisar Model School & College. The document has been produced by reverse-engineering an existing Django/Python implementation of the system, and is intended to allow a developer or team to rebuild the application from scratch, under the school's own name and branding, with equivalent functionality, data model, and business rules.

1.2 Document Conventions

- FR-x.y denotes a Functional Requirement in module x, item y.
- Model field tables list the exact database columns, types, and constraints required.
- Text in this document reflects the behaviour actually implemented in the source project, including edge cases and auto-generated values.

1.3 Intended Audience

- Software developers rebuilding or extending the system (Django/Python stack assumed, per user preference).
- Project managers and QA engineers who need a functional baseline for test-case design.
- System administrators / school owners who need to understand what the software does operationally.

1.4 Project Scope

KMS is a single-school (single-tenant) administrative system covering four operational domains: (1) Student Information Management including admissions, promotions, withdrawals and family/household grouping; (2) Fee & Finance Management including per-student monthly fee ledgers, arrears roll-over, receipts and defaulter tracking; (3) Teacher/HR Management including salary scales and monthly payroll bill generation; and (4) General Accounts including expense recording and a consolidated cashbook (income vs. expense) with daily/monthly summaries. The system also produces printable certificates (School Leaving Certificate, Character Certificate, Staff Experience Certificate, Staff ID Card, Fee Receipts, Salary Slips) and provides a dashboard with key operational metrics. Authentication uses Django's built-in session-based auth; there is no public self-service portal — every screen requires a logged-in staff user.

1.5 Definitions, Acronyms and Abbreviations

Field	Type	Constraints / Description
KMS	General term	Kohisar Model School & College Management System — the branded product name for this rebuild (school logo and name applied throughout the UI, login page, receipts and certificates); the underlying Django project/folder may keep the short technical name "KMS" (Django project: core).

Field	Type	Constraints / Description
SRS	General term	Software Requirements Specification.
Ledger	Domain term	A StudentFeeLedger — one per (student, academic session) pair, containing 12 monthly fee entries.
Arrears	Domain term	Unpaid balance carried forward from the previous month's fee entry into the current month.
Session	Domain term	AcademicSession — a school year, e.g. "2025-2026", with a start date, end date and an is_active flag.
FamilyHousehold	Domain term	A group of sibling students who share a guardian/household and can be billed together via a single family payment.
SLC	Domain term	School Leaving Certificate — issued to a withdrawn student.

2. Overall Description

2.1 Product Perspective

KMS is a standalone, server-rendered web application built on the Django framework (Python). It is not part of a larger product family and has no external system integrations in its current form (no SMS/email gateway, no payment gateway, no biometric attendance). All data is stored in a local relational database (SQLite in the reference implementation) and all pages are rendered server-side with Bootstrap 5 styling via django-crispy-forms. The Django REST Framework package is installed but not currently used by any exposed API endpoint in the reference build — it is reserved for future API expansion.

This rebuild is branded for Kohisar Model School & College. Branding is entirely data-driven through the existing SchoolSetting model (Section 5.1) and the global school_context context processor (Section 3), so no template code needs to be hard-coded with the school's identity: the school name, address, contact and logo image populate the sidebar header, the login page, every printable document (receipts, salary slips, ID cards, certificates) and the dashboard simply by seeding one SchoolSetting row with the school's real name and logo file — see FR-2.1 and Section 8, rule 11.

2.2 Product Functions (Summary)

- Staff login/logout using Django's built-in authentication views.
- School profile configuration: name, address, logo, contact, academic session numbering, receipt numbering.
- Academic session, class level and section (class + section, e.g. "Class 5 - A") management.
- Student admission with auto-generated admission numbers, profile photos, and family linkage.
- Family/household management for grouping siblings and enabling combined fee payments.
- Searchable/filterable student list, admission register and withdrawal register.
- Single-student and bulk class/section promotion between academic sessions, with promotion history logging.
- Student withdrawal workflow that freezes future fee charges and records leaving details.
- Automatic 12-month fee ledger generation per student per session, pro-rated for admission/withdrawal month.
- Fee collection (single student or whole family) with automatic allocation of a lump-sum payment across the oldest unpaid months first, auto-generated receipt numbers, and month-by-month arrears roll-forward.
- Manual fee adjustments (change monthly fee going forward, edit exam fee / other charges for a specific month).
- Financial reports: fee register grid, class-wise fee summary, family dues summary, defaulters list.
- Teacher/staff records with salary scale assignment.
- Monthly salary bill batch-generation for all active teachers, with manual editing, payment marking and voucher numbers.
- General expense recording by category and payment mode.

- Consolidated cashbook combining fee income, expenses and paid salaries into daily and monthly income/expense/net summaries.
- Printable documents: fee receipt, salary slip, School Leaving Certificate, Character Certificate, Staff Experience Certificate, Staff ID Card.
- Dashboard showing active student/teacher counts, current-month fee collection, current-month expenses and recent receipts.
- One-click SQLite database backup download.

2.3 User Classes and Characteristics

Field	Type	Constraints / Description
School Administrator / Superuser	Full access	Manages school settings, academic sessions, classes/sections, salary scales, and has Django admin access to every model.
Office / Accounts Staff	Operational access	Day-to-day use: admissions, fee collection, expense entry, salary bill processing, printing certificates and receipts. All application views only require <code>request.user.is_authenticated</code> — the reference implementation does not yet differentiate staff by role/group beyond Django's <code>is_staff/is_superuser</code> for <code>/admin/</code> access.

Note: The reference codebase enforces login (`@login_required`) on every business view but does not implement per-role permission checks (e.g. Django Groups/Permissions) beyond what Django's admin site provides natively. A rebuild that needs role separation (e.g. "Accountant cannot edit students") should add this as a new requirement — see Section 8.

2.4 Operating Environment

- Server: Python 3.x running Django 6.0.1 (as declared in `settings.py`'s docstring).
- Database: SQLite 3 (file-based, `db.sqlite3`) — swappable for PostgreSQL/MySQL via Django's `DATABASES` setting without code changes.
- Web server: Django development server for local use; a production deployment would sit behind Gunicorn/uWSGI + Nginx (not configured in the reference build).
- Client: Any modern desktop or tablet web browser (Chrome, Edge, Firefox). UI uses Bootstrap 5 + Font Awesome and is best used on screens $\geq 992\text{px}$ due to data-dense tables.
- Timezone: Asia/Karachi (fixed in settings).

2.5 Design and Implementation Constraints

- Django MVT (Model-View-Template) architecture with function-based views throughout (no class-based views, no DRF viewsets in use).
- `django-crispy-forms` with the `bootstrap5` template pack is used to render all forms consistently.
- Media files (student/teacher photos, school logo) are stored on local disk under `/media/` during development (served via Django `static/media` helpers when `DEBUG=True`).

- All monetary fields use `DecimalField(max_digits=10, decimal_places=2)` — no floating point currency math.
- Single-currency, single-school (single-tenant) design — there is no multi-branch/multi-school data partitioning.
- `SECRET_KEY`, `DEBUG=True` and empty `ALLOWED_HOSTS` in the reference `settings.py` are development-only values and MUST be replaced for any production deployment (see Section 7.2).

2.6 Assumptions and Dependencies

- A single `AcademicSession` is marked `is_active=True` at any time; business logic (fee register default, defaulters, dues, class summary) falls back to "first session" if none is active.
- Exactly one `SchoolSetting` row is expected to exist (used for the school profile, logo and running receipt-number counter); the receipt numbering logic assumes a single row.
- Class monthly fee (`ClassLevel.monthly_fee`) is the default fee applied when a ledger is initialized; it can subsequently be overridden per month via the Fee Adjustment feature.
- It is assumed office staff will run "Generate Salary Bills" once per month and "Bulk Promote" once per academic year.

3. System Architecture

The system is organised as a Django project (core) composed of five local Django apps, each owning a distinct bounded context, plus Django's built-in auth app for login/logout:

Field	Type	Constraints / Description
core	Project shell	Global settings, root URL configuration, WSGI/ASGI entry points, and a context processor (school_context) that injects the active academic session and school settings into every template.
school	App	School profile (SchoolSetting), AcademicSession, ClassLevel, Section. Also owns the dashboard view and the DB backup-download utility.
students	App	Student, FamilyHousehold, PromotionHistory. Admissions, listing/search, promotion, withdrawal, family CRUD, and (via a sub-module) certificate rendering (SLC, Character Certificate).
teachers	App	SalaryScale, Teacher, MonthlySalaryBill. Staff CRUD, salary-bill batch generation and editing, experience certificate and staff ID card printing.
finance	App	StudentFeeLedger, StudentFeeMonthEntry, FeePaymentReceipt, plus fee business-logic utilities (finance/utls.py). Fee register, fee collection (individual & family), fee adjustment, receipts, defaulters, class & family dues summaries.
accounts	App	Expense. Expense entry and the consolidated cashbook (daily/monthly income vs expense summaries). Note: this app is unrelated to Django's authentication "accounts" — login/logout is handled by django.contrib.auth.urls, not this app.

Inter-app dependencies (import direction): students imports school; finance imports students and school; teachers is independent (only imports school for the school profile in certificate views); accounts imports finance and teachers (to fold fee income and paid salaries into the cashbook); school imports students, teachers, finance and accounts only inside the dashboard view.

4. Functional Requirements

4.1 Authentication & Access Control

ID	Requirement Description
FR-1.1	The system shall provide a login page (custom-styled template) using Django's built-in authentication view (django.contrib.auth.urls), accepting username and password.
FR-1.2	On successful login, the system shall redirect the user to the Dashboard (LOGIN_REDIRECT_URL = 'dashboard').
FR-1.3	The system shall provide logout, redirecting to the login page (LOGOUT_REDIRECT_URL = 'login').
FR-1.4	Every business page (students, finance, teachers, accounts, dashboard) shall require an authenticated session; unauthenticated requests shall be redirected to the login page (LOGIN_URL = 'login').
FR-1.5	The Django admin site (/admin/) shall remain available for superusers to manage all models directly, including bulk data corrections.

4.2 School Configuration Module

ID	Requirement Description
FR-2.1	The system shall store one School Setting record containing: school name, address, logo image, contact info, academic-session name format (default "2025-2026"), a receipt-number prefix (default "REC-") and the last-issued receipt sequence number (default 1000). For this deployment, the row shall be seeded with name = "Kohisar Model School & College" and the school's circular crest logo, so that the name and logo shown in the reference screenshot (sidebar, dashboard header) appear identically across the sidebar, login page, dashboard, and every printable document.
FR-2.2	The system shall allow creation of multiple Academic Sessions, each with a unique name, start date, end date and an is_active flag; exactly one session is expected to be flagged active at a time (enforced by admin operator discipline, not a database constraint).
FR-2.3	The system shall allow creation of Class Levels, each with a unique name, a level category (Prep / KG / Primary 1-5 / Middle 6-8 / High 9-10 / Higher Secondary 11-12) and a default monthly fee.
FR-2.4	The system shall allow creation of Sections under a Class Level (e.g. "5 - A"), each with an optional class-teacher name; the (class_level, name) pair shall be unique.
FR-2.5	The active academic session and the school setting shall be available to every page via a global template context processor for use in headers/branding.

4.3 Student Information Module

ID	Requirement Description
FR-3.1	The system shall allow adding a new student with: admission number (auto-generated if left blank), admission date, full name, father's name, date of birth, gender, tribe/caste,

ID	Requirement Description
	father's occupation, residence address, contact number, profile picture, current class, current section, optional family link, status and remarks.
FR-3.2	Auto-numbering rule: if no admission number is supplied, the system shall take the highest existing purely-numeric admission number, add 1, starting from 1001 if no students exist, and shall keep incrementing past any pre-existing collision until a free number is found.
FR-3.3	The system shall provide a searchable, filterable student list (filters: class, section, status; free-text search across full name, admission number, father's name and residence).
FR-3.4	The system shall provide a student detail page and an edit form for all admission fields.
FR-3.5	When a student's status is changed to "Inactive" and an inactive date is set, the system shall locate the fee ledger for the academic session containing that date (or the active session as fallback), freeze future months' charges from that date forward, and recalculate arrears roll-forward for the remaining months.
FR-3.6	The system shall support single-student promotion to a target class, section and academic session; the target section must belong to the target class (validated) or the request shall be rejected with a form error.
FR-3.7	The system shall support bulk promotion: staff select a source class + section, view its Active students, multi-select students, and promote them together to a target class, section and session in one transaction.
FR-3.8	Withdrawn students shall not be eligible for promotion (single or bulk).
FR-3.9	Every promotion (single or bulk) shall be recorded in a Promotion History log capturing the student, from-class, from-section, to-class, to-section, target session name and promotion date (auto timestamp).
FR-3.10	On promotion into a session for which the student has no existing fee ledger, the system shall auto-create a new 12-month ledger for that session using the destination class's monthly fee, then recompute arrears.
FR-3.11	The system shall provide a formal Withdrawal workflow capturing withdrawal date, class-at-withdrawal (auto-filled from current class/section if blank), arrears-at-withdrawal (pre-filled by summing outstanding balances up to the current month, if available) and a remarks/reason field; on submit the student status shall be set to "Withdrawn".
FR-3.12	On withdrawal, the system shall locate the relevant session's fee ledger, zero out monthly fee / exam fee / other charges for all months after the withdrawal month, and recalculate arrears.
FR-3.13	The system shall provide an Admission Register (all students ordered by admission date, searchable) and a Withdrawal Register (Withdrawn students only, ordered by withdrawal date, searchable) for print/record purposes.

4.4 Family / Household Module

ID	Requirement Description
FR-4.1	The system shall allow grouping students who share a guardian into a Family Household record: auto-generated Family ID (format FAM-0001, incrementing, collision-safe), guardian name, contact number and address.
FR-4.2	The system shall provide a searchable family list (search across family ID, guardian name, contact, address) and family add/edit forms.
FR-4.3	A student may optionally be linked to one Family Household (nullable foreign key); if the family is deleted the student's family link shall be set to null rather than the student being deleted.

4.5 Fee & Finance Management Module

ID	Requirement Description
FR-5.1	The system shall maintain one Fee Ledger per (student, academic session), containing exactly 12 Monthly Fee Entries (Jan–Dec), each with: arrears, monthly fee, exam fee, other charges, paid amount, paid flag, last-payment date and last-receipt-number.
FR-5.2	Ledger initialization shall set each month's monthly fee to the class's default fee, except: months before the admission month within the session are zeroed, and (if applicable) months after a withdrawal month within the session are zeroed.
FR-5.3	The system shall expose a Fee Register grid view: rows = active students (optionally filtered by class), columns = the 12 months, filterable by academic session, defaulting to the active session.
FR-5.4	The system shall provide a Collect Fee screen per student: staff select one or more unpaid/partially-paid month entries and enter an amount paid. The amount must be > 0 and must not exceed the sum of outstanding balances for the selected months, or the form shall reject it with a validation error.
FR-5.5	On a valid fee collection, the system shall create a Fee Payment Receipt (auto-numbered using the school's receipt prefix + incrementing counter, e.g. REC-1001, wrapped in a database transaction to avoid duplicate numbers), link it to the selected month entries, and allocate the paid amount across those months strictly in month order — each month's balance is paid off before moving to the next — recording last payment date and receipt number on each fully/partially paid entry.
FR-5.6	If a payment leaves an unallocated remainder (should not normally happen given the validation in FR-5.4, but guarded regardless), the system shall append an "[Unallocated: amount]" note to the receipt's remarks.
FR-5.7	After any payment, the system shall recalculate the ledger's arrears roll-forward: for month 2..12, each month's arrears equals the prior month's outstanding balance; each month's paid flag is refreshed.
FR-5.8	The system shall provide a Family Payment screen: staff select a Family Household, and pay across selected month entries belonging to any of that family's linked students in one receipt; the receipt shall be tagged in its remarks with the family ID and linked to the family's first student for searchability; arrears shall be recalculated for every affected ledger.

ID	Requirement Description
FR-5.9	The system shall provide a printable Fee Receipt view showing receipt number, date, amount, covered months/students and school header/logo.
FR-5.10	The system shall provide a Fee Adjustment screen per student (active session) allowing: changing the monthly fee for a given month and all subsequent months in the same ledger, and/or editing exam fee / other charges for a single specific month; arrears shall be recalculated afterward.
FR-5.11	The system shall provide a Defaulters List: for the active (or first) session, list every active student with at least one month entry whose balance (arrears + monthly fee + exam fee + other charges – paid amount) is greater than zero, aggregated per student as total dues and count of unpaid months; filterable by class and section.
FR-5.12	The system shall provide a Family Dues summary: total outstanding balance per family household, summed across all of that family's students' unpaid month entries in the active session, sorted highest-due first.
FR-5.13	The system shall provide a Class Fee Summary report: for every class level, total amount due, total amount paid and outstanding balance for the active session.

4.6 Teacher & Payroll Module

ID	Requirement Description
FR-6.1	The system shall allow defining Salary Scales, each with a unique name, basic pay, medical allowance, conveyance allowance and other allowances.
FR-6.2	The system shall allow adding Teacher/staff records with: teacher ID, full name, father's name, CNIC, date of birth, qualification, experience, contact number, address, joining date, designation, an optional salary scale, profile picture, status (Active/Inactive) and remarks.
FR-6.3	The system shall provide a staff list and a staff detail page showing the teacher's profile plus their full salary-bill history (most recent first).
FR-6.4	The system shall provide a batch "Generate Salary Bills" action for a chosen month/year: for every Active teacher who has a salary scale assigned, create a Monthly Salary Bill (if one does not already exist for that teacher/month/year — enforced by a unique constraint) using base pay = scale's basic pay and allowances = sum of the scale's medical + conveyance + other allowances; report how many bills were newly created.
FR-6.5	Each Monthly Salary Bill shall record: month, year, base pay, days present (default 30), allowances, deductions, paid flag, payment date and voucher number; net payable is computed as (base pay + allowances) – deductions.
FR-6.6	The system shall allow editing an individual salary bill (e.g. adjusting days present, allowances, deductions, marking as paid); if marked paid without an explicit payment date, the system shall default the payment date to today.
FR-6.7	The system shall provide a salary-bill list filterable by month and year, and a printable Salary Bill / Slip detail view including the school header.
FR-6.8	The system shall provide printable Staff documents: an Experience Certificate and a Staff ID Card, both including the school profile/logo.

4.7 Accounts / Cashbook Module

ID	Requirement Description
FR-7.1	The system shall allow recording an Expense with: date, category (Stationery / Maintenance / Electricity / Salary / Other), amount, optional note, optional reference number and payment mode (Cash / Bank).
FR-7.2	The system shall provide a consolidated Cashbook view combining: all fee receipts (income), all expenses, and all paid salary bills' net-payable amounts (treated as expenses); optionally filtered by a start/end date range; showing total income, total expenses (including paid salaries) and the resulting net balance.
FR-7.3	The system shall provide a Daily Summary report: for every date on which there was any fee income, expense or salary payment, show total income, total expenses and net for that day, most recent first.
FR-7.4	The system shall provide a Monthly Summary report: the same income/expense/net rollup grouped by calendar month, most recent first.

4.8 Certificates & Printable Documents

ID	Requirement Description
FR-8.1	The system shall generate a School Leaving Certificate for any student, populated with student and school-profile data, formatted for printing.
FR-8.2	The system shall generate a Character Certificate for any student, formatted for printing.
FR-8.3	The system shall generate a Staff Experience Certificate and a Staff ID Card for any teacher, formatted for printing.
FR-8.4	The system shall generate a printable Fee Receipt and a printable Salary Bill/Slip, each including the school's logo, name and address pulled from School Setting.

4.9 Dashboard & Reporting

ID	Requirement Description
FR-9.1	On login, the Dashboard shall display: count of Active students, count of Active teachers, total fee amount collected in the current calendar month, total expenses recorded in the current calendar month, and the 5 most recent fee receipts.
FR-9.2	The system shall provide a one-click database backup: download the live SQLite database file with a timestamped filename (e.g. KMS_Backup_20260904_1230.sqlite3).

5. Data Model (Database Design)

The following tables define every model required to rebuild the system's database schema exactly. Unless noted otherwise, every model uses an auto-incrementing BigAutoField primary key (id).

5.1 school app

SchoolSetting

Field	Type	Constraints / Description
name	CharField(255)	Required. School's display name.
address	TextField	Required.
logo	ImageField	Optional. Uploaded to school_logos/.
contact	CharField(100)	Required.
academic_session_format	CharField(50)	Default "2025-2026".
receipt_no_prefix	CharField(10)	Default "REC-". Used to build receipt numbers.
last_receipt_no	IntegerField	Default 1000. Incremented atomically each time a receipt is issued.

AcademicSession

Field	Type	Constraints / Description
name	CharField(20), unique	e.g. "2025-2026".
start_date	DateField	Required.
end_date	DateField	Required.
is_active	BooleanField	Default False. Only one session should be active at a time (business rule, not DB-enforced).

ClassLevel

Field	Type	Constraints / Description
name	CharField(50), unique	e.g. "Class 5".
level	CharField(20), choices	Prep / KG / Primary(1-5) / Middle(6-8) / High(9-10) / Higher Secondary(11-12).
monthly_fee	Decimal(10,2)	Default 0.00. Default monthly fee applied when a ledger is created for this class.

Section

Field	Type	Constraints / Description
class_level	FK → ClassLevel, CASCADE	related_name='sections'.

Field	Type	Constraints / Description
name	CharField(10)	e.g. A, B, C. Unique together with class_level.
class_teacher	CharField(255)	Optional free-text teacher name.

5.2 students app

FamilyHousehold

Field	Type	Constraints / Description
family_id	CharField(20), unique	Auto-generated on save if blank: FAM-0001, FAM-0002 ... collision-checked.
father_guardian_name	CharField(255)	Required.
contact_number	CharField(20)	Required.
address	TextField	Required.

Student

Field	Type	Constraints / Description
admission_no	CharField(20), unique	Auto-generated on save if blank: highest existing numeric admission_no + 1, starting at 1001, collision-checked.
admission_date	DateField	Required.
full_name	CharField(255)	Required.
father_name	CharField(255)	Required.
dob	DateField	Required.
gender	CharField(10), choices	Male / Female / Other. Default Male.
tribe_caste	CharField(100)	Optional.
father_occupation	CharField(100)	Optional.
residence	TextField	Required.
contact_number	CharField(20)	Required.
profile_picture	ImageField	Optional. Uploaded to students/.
current_class	FK → ClassLevel, PROTECT	related_name='students'. Cannot delete a class in use.
current_section	FK → Section, PROTECT	related_name='students'.
family	FK → FamilyHousehold, SET_NULL, nullable	related_name='students'. Optional.
status	CharField(15), choices	Active / Inactive / Withdrawn. Default Active.

Field	Type	Constraints / Description
inactive_date	DateField	Optional. Set when status becomes Inactive.
remarks	TextField	Optional.
withdrawal_date	DateField	Optional. Set on formal withdrawal.
class_at_withdrawal	CharField(50)	Optional. Auto-filled from current class/section if blank.
arrears_at_withdrawal	Decimal(10,2)	Default 0.00.

PromotionHistory

Field	Type	Constraints / Description
student	FK → Student, CASCADE	related_name='promotions'.
from_class / to_class	FK → ClassLevel, PROTECT (x2)	related_name='promotions_from' / 'promotions_to'.
from_section / to_section	FK → Section, PROTECT (x2)	related_name='promotions_from' / 'promotions_to'.
session	CharField(20)	Target session's name, stored as text (snapshot).
promotion_date	DateField, auto_now_add	Set automatically to the date of the promotion.

5.3 teachers app

SalaryScale

Field	Type	Constraints / Description
name	CharField(100), unique	e.g. "Grade 17", "PST Scale".
basic_pay	Decimal(10,2)	Required.
medical_allowance	Decimal(10,2)	Default 0.00.
conveyance_allowance	Decimal(10,2)	Default 0.00.
other_allowances	Decimal(10,2)	Default 0.00.

Teacher

Field	Type	Constraints / Description
teacher_id	CharField(20), unique	Manually assigned (no auto-generation logic in reference views).
full_name	CharField(255)	Required.
father_name	CharField(255)	Optional.
cnic	CharField(20)	Optional.
dob	DateField	Required.

Field	Type	Constraints / Description
qualification	TextField	Required.
experience	TextField	Required.
contact_number	CharField(20)	Required.
address	TextField	Required.
joining_date	DateField	Required.
designation	CharField(100)	e.g. SST, PST.
salary_scale	FK → SalaryScale, SET_NULL, nullable	Optional.
profile_picture	ImageField	Optional. Uploaded to teachers/.
status	CharField(10), choices	Active / Inactive. Default Active.
remarks	TextField	Optional.

MonthlySalaryBill

Field	Type	Constraints / Description
teacher	FK → Teacher, CASCADE	related_name='salary_bills'.
month	IntegerField	1–12.
year	IntegerField	e.g. 2026.
base_pay	Decimal(10,2)	Snapshot of the scale's basic pay at generation time.
days_present	IntegerField	Default 30.
allowances	Decimal(10,2)	Default 0.00. Snapshot sum of scale allowances at generation time.
deductions	Decimal(10,2)	Default 0.00.
is_paid	BooleanField	Default False.
payment_date	DateField	Optional; auto-set to today if marked paid without a date.
voucher_no	CharField(50)	Optional.
net_payable	Computed property	(base_pay + allowances) – deductions. Not a stored column.
— constraint —	unique_together	(teacher, month, year) — one bill per teacher per month.

5.4 finance app

StudentFeeLedger

Field	Type	Constraints / Description
student	FK → Student, CASCADE	related_name='fee_ledgers'.
academic_session	FK → AcademicSession, PROTECT	—
created_at	DateTimeField, auto_now_add	—
— constraint —	unique_together	(student, academic_session) — one ledger per student per session.

StudentFeeMonthEntry

Field	Type	Constraints / Description
ledger	FK → StudentFeeLedger, CASCADE	related_name='monthly_entries'.
month	IntegerField, choices	1 (January) – 12 (December).
arrears	Decimal(10,2)	Default 0.00. Set to prior month's balance during recalculation.
monthly_fee	Decimal(10,2)	Default 0.00.
exam_fee	Decimal(10,2)	Default 0.00.
other_charges	Decimal(10,2)	Default 0.00.
paid_amount	Decimal(10,2)	Default 0.00. Accumulates across one or more receipts.
is_paid	BooleanField	Default False. True when paid_amount ≥ total_amount.
last_payment_date	DateField	Optional.
last_receipt_no	CharField(50)	Optional. Copies the most recent covering receipt's number.
total_amount	Computed property	arrears + monthly_fee + exam_fee + other_charges.
balance	Computed property	total_amount – paid_amount.
— constraint —	unique_together	(ledger, month). Ordered by month.

FeePaymentReceipt

Field	Type	Constraints / Description
receipt_no	CharField(50), unique	Auto-generated on save if blank, using SchoolSetting.receipt_no_prefix + an atomically-incremented SchoolSetting.last_receipt_no; falls

Field	Type	Constraints / Description
		back to "REC-" + random hex if no SchoolSetting row exists.
payment_date	DateField	Required.
amount_paid	Decimal(10,2)	Required.
month_entries	M2M → StudentFeeMonthEntry	related_name='payments'. All month entries this single receipt paid towards (spans multiple students for family payments).
student	FK → Student, PROTECT	related_name='payments'. Primary/searchable student reference (first sibling for family payments).
remarks	TextField	Optional. Used to tag family payments and note unallocated remainders.

5.5 accounts app

Expense

Field	Type	Constraints / Description
date	DateField	Required.
category	CharField(50), choices	Stationery / Maintenance / Electricity / Salary / Other.
amount	Decimal(10,2)	Required.
note	TextField	Optional.
reference	CharField(50)	Optional.
payment_mode	CharField(20), choices	Cash / Bank. Default Cash.

5.6 Entity Relationship Summary

Key relationships to preserve when rebuilding:

- FamilyHousehold 1—N Student (nullable; on delete, student.family → NULL).
- ClassLevel 1—N Section; ClassLevel/Section 1—N Student (PROTECT — cannot delete a class/section that has students).
- Student 1—N PromotionHistory, 1—N StudentFeeLedger, 1—N FeePaymentReceipt (as primary student).
- AcademicSession 1—N StudentFeeLedger (PROTECT).
- StudentFeeLedger 1—12 StudentFeeMonthEntry.
- StudentFeeMonthEntry M—N FeePaymentReceipt (a receipt can span several months/students; a month entry can be paid via more than one receipt if paid in installments).
- SalaryScale 1—N Teacher (nullable, SET_NULL); Teacher 1—N MonthlySalaryBill.

6. External Interface Requirements

6.1 User Interfaces

- A persistent left sidebar for primary navigation (Dashboard, Students, Families, Fee Register, Defaulters, Teachers, Salary Bills, Cashbook), collapsible via a top navbar toggle.
- Bootstrap 5 styling with Font Awesome icons throughout; forms rendered via django-crispy-forms (bootstrap5 pack) for consistent field layout and validation-error display.
- The login page hides the sidebar/navbar and presents a centered card on a dark gradient background.
- Printable pages (receipts, salary slips, certificates, ID cards, registers) use dedicated templates suitable for browser "Print to PDF"/physical printing, including the school logo and address pulled from School Setting.
- List/report screens (student list, fee register, defaulters, family dues, class summary, salary bill list, cashbook) provide filter controls (class, section, status, session, month, year, date range) as GET query parameters, plus free-text search where noted in Section 4.

6.2 Hardware Interfaces

None specific. Standard server hardware capable of running Python/Django; client-side, any device with a modern web browser and (optionally) a printer for certificates/receipts.

6.3 Software Interfaces

Field	Type	Constraints / Description
Django	6.0.1	Core web framework (MVT).
djangoestframework	installed	Present in INSTALLED_APPS for future API use; not used by any current endpoint.
django-crispy-forms	installed	Form rendering.
crispy-bootstrap5	installed	Bootstrap 5 template pack for crispy-forms (CRISPY_TEMPLATE_PACK='bootstrap5').
SQLite3	file-based	Default database engine; swappable via DATABASES setting.
Bootstrap 5 / Font Awesome	CDN or static	Front-end styling.

6.4 Communications Interfaces

Standard HTTP/HTTPS. No third-party API, SMS, email, or payment-gateway integrations exist in the reference implementation.

7. Non-Functional Requirements

7.1 Performance

- Report views that aggregate across all students (defaulters, class summary, family dues) should use database-level aggregation (Django ORM `F()/Sum()/ExpressionWrapper`, as in the reference implementation) rather than looping in Python, to remain responsive as student counts grow.
- List views should support pagination if the school's student/teacher count grows beyond a few hundred records (not implemented in the reference build — recommended addition for a rebuild).

7.2 Security

- Production deployments MUST set `DEBUG=False`, define a real `ALLOWED_HOSTS` list, and replace `SECRET_KEY` with a securely generated, environment-sourced value — all three are development-only placeholders in the reference `settings.py`.
- All state-changing views must remain behind `@login_required` (or equivalent) as in the reference implementation.
- CSRF protection (Django's default `CsrfViewMiddleware`) must be preserved on all forms.
- Passwords are managed by Django's default auth password hashers and validators (similarity, minimum length, common-password and numeric-password validators are all enabled).
- Consider adding role-based permissions (see Section 2.3 note) if multiple staff roles need different access levels.

7.3 Reliability & Data Integrity

- Receipt-number generation and salary-bill batch creation must be wrapped in database transactions (`transaction.atomic`, as in the reference code) to avoid duplicate numbers or partial batch writes under concurrent use.
- Foreign keys to `ClassLevel/Section/AcademicSession` from records that must not be orphaned (`Student`, `StudentFeeLedger`) use `PROTECT` to block deletion of in-use reference data.
- `unique_together` / `unique` constraints (`admission_no`, `family_id`, `teacher_id`, `receipt_no`, (`student`, `session`) ledgers, (`ledger`, `month`) entries, (`teacher`, `month`, `year`) bills, (`class_level`, `name`) sections) must be preserved to prevent duplicate records.

7.4 Usability

- Auto-numbering (admission numbers, family IDs, receipt numbers) removes manual sequence-tracking error from data-entry staff.
- Pre-filled defaults (e.g. withdrawal class/arrears, today's date on new forms) reduce repetitive data entry.
- Consistent flash messaging (Django messages framework) confirms every successful create/update/payment action.

7.5 Maintainability

- Each business domain is isolated in its own Django app with its own models/views/urls/forms/admin, keeping the codebase modular and independently testable.
- Fee arithmetic (roll-over, initialization, withdrawal stop) is centralised in finance/utls.py rather than duplicated across views.

7.6 Portability

- Swapping the database engine (e.g. to PostgreSQL) requires only a DATABASES setting change, since no raw SQL is used.
- Media/static file storage can be moved to cloud storage (e.g. S3) via Django's storage backend abstraction without changing application logic.

8. Business Rules Summary

These are the calculation/workflow rules most critical to reproduce exactly, consolidated from Section 4 for quick developer reference:

1. Admission number = (highest existing numeric admission_no) + 1, starting at 1001; skip forward past any collision.
2. Family ID = FAM-<sequential 4-digit number>, based on (last family's internal id) + 1; skip forward past any collision.
3. Receipt number = SchoolSetting.receipt_no_prefix + (SchoolSetting.last_receipt_no incremented by 1 atomically); fallback to a random "REC-XXXXXX" hex code if no SchoolSetting row exists.
4. A new fee ledger's 12 monthly entries default to the class's monthly_fee, except: months strictly before the admission month (if admission falls inside the session) are 0.00, and months strictly after a withdrawal month (if applicable) are 0.00.
5. Arrears roll-forward: month 1's arrears is whatever was set at initialization (0.00 unless later adjusted); for months 2–12, arrears = previous month's balance (total_amount – paid_amount), recomputed in month order every time recalculate_ledger_arrears runs.
6. A fee payment (individual or family) is validated so that amount_paid is > 0 and ≤ the sum of outstanding balances on the selected month entries; it is then applied to the selected entries in ascending month order, fully paying off each month's balance before spilling into the next, until the amount is exhausted.
7. Changing the monthly fee via Fee Adjustment for month M applies that new fee to month M and every subsequent month (month__gte=M) in the same ledger; exam fee / other charges adjustments apply to the single specified month only.
8. Marking a student Inactive (with an inactive_date) or Withdrawn (with a withdrawal_date) zeroes monthly_fee, exam_fee and other_charges for every month after that date's month within the applicable session's ledger, then recalculates arrears.
9. Salary-bill generation only creates a bill for teachers who are Active and have a salary_scale assigned; it will not duplicate a bill for a (teacher, month, year) that already exists.
10. The cashbook and daily/monthly summaries treat fee receipts as income and both general Expenses and paid MonthlySalaryBill.net_payable amounts as outgoings.
11. Branding is centralised: the sidebar logo/name, login page, dashboard header, and every printable document (receipt, salary slip, SLC, character certificate, experience certificate, staff ID card) render the single seeded SchoolSetting row's name/logo — for this deployment "Kohisar Model School & College" and its crest — so a future re-brand only requires updating that one row, never template code.

9. Appendix A — URL / Route Map

Root URLconf (core/urls.py) mounts each app as follows: /admin/ → Django admin · /accounts/ → django.contrib.auth.urls (login/logout/password views) · /students/ → students.urls · /finance/ → finance.urls · /teachers/ → teachers.urls · /cashbook/ → accounts.urls · / → school.urls.

school.urls (mounted at /)

Path	View	Notes
/	dashboard	GET
/backup/	download_db_backup	GET — downloads db.sqlite3

students.urls (mounted at /students/)

Path	View	Notes
/	student_list	list + filters + search
/add/	student_create	GET/POST
/admission-register/	admission_register	GET
/withdrawals/	withdrawal_register	GET
/bulk-promote/	bulk_promote	GET/POST
/families/	family_list	GET
/families/add/	family_create	GET/POST
/families/<pk>/edit/	family_update	GET/POST
/<pk>/	student_detail	GET
/<pk>/edit/	student_update	GET/POST
/<pk>/promote/	student_promote	GET/POST
/<pk>/slc/	school_leaving_certificate	GET — printable
/<pk>/character/	character_certificate	GET — printable
/<pk>/withdraw/	student_withdraw	GET/POST

teachers.urls (mounted at /teachers/)

Path	View	Notes
/	teacher_list	GET
/add/	teacher_create	GET/POST
/<pk>/	teacher_detail	GET
/<pk>/edit/	teacher_update	GET/POST
/<pk>/experience/	experience_certificate	GET — printable
/<pk>/id-card/	staff_id_card	GET — printable
/salary-bills/	salary_bill_list	GET, filter by month/year

Path	View	Notes
/salary-bills/<pk>/	salary_bill_detail	GET — printable slip
/salary-bills/<pk>/edit/	salary_bill_update	GET/POST
/salary-bills/generate/	generate_salary_bills	GET/POST — batch

finance.urls (mounted at /finance/)

Path	View	Notes
/register/	fee_register	GET — grid, filter by session/class
/collect/<student_id>/	collect_fee	GET/POST
/family-payment/<household_id>/	family_payment	GET/POST
/adjust/<student_id>/	fee_adjust	GET/POST
/receipt/<pk>/	fee_receipt	GET — printable
/family-dues/	family_dues	GET
/class-summary/	class_fee_summary	GET
/defaulters/	defaulters_list	GET, filter by class/section

accounts.urls (mounted at /cashbook/)

Path	View	Notes
/	cashbook_summary	GET, filter by start/end date
/add-expense/	add_expense	GET/POST
/daily/	daily_summary	GET
/monthly/	monthly_summary	GET

10. Appendix B — Technology Stack & Setup

10.1 Recommended Stack for Rebuild

- Backend: Python 3.11+, Django 5.x/6.x
- Packages: djangorestframework, django-crispy-forms, crispy-bootstrap5, Pillow (required for ImageField uploads — not listed in the reference INSTALLED_APPS but required by Django for image fields)
- Database: SQLite for development; PostgreSQL recommended for production
- Frontend: Bootstrap 5 + Font Awesome (CDN), server-rendered Django templates

10.2 Suggested requirements.txt

```
Django>=6.0,<7.0 djangorestframework django-crispy-forms crispy-bootstrap5 Pillow
```

10.3 Suggested Rebuild Order

12. Scaffold the core project and the five apps (school, students, teachers, finance, accounts); wire INSTALLED_APPS, crispy-forms settings, auth redirect URLs, and the school_context context processor.
13. Implement school app models/admin first (SchoolSetting, AcademicSession, ClassLevel, Section) — everything else depends on them.
14. Implement students app (FamilyHousehold, Student, PromotionHistory) including the auto-numbering save() overrides.
15. Implement teachers app (SalaryScale, Teacher, MonthlySalaryBill).
16. Implement finance app models plus finance/utils.py (initialize_ledger_months, recalculate_ledger_arrears, apply_withdrawal_stop, roll_over_arrears) before wiring any finance views — the views depend heavily on these helpers.
17. Implement accounts app (Expense) and the cashbook views, which read from both finance and teachers.
18. Build templates last, starting from base.html (sidebar/navbar shell) and the login page, then one module at a time.
19. Seed a School Setting row and at least one Academic Session (is_active=True) before testing fee flows — both are assumed to exist by the finance views.